Predicting Prompt-Injection Compliance from LLM Internal Activations

Noah Safar · Advisor: Timos Antonopoulos

CPSC 4900: Senior Project · Fall 2026 · September 17, 2026

1 Introduction

Large language models are increasingly deployed in settings where they process content that the user did not supply, such as retrieved web pages, emails, documents, and the outputs of tools they call. This exposes them to *prompt injection*, in which the model wrongly interprets text embedded in that untrusted content as an instruction rather than data. The variant that matters in deployment is indirect prompt injection, where the malicious instruction arrives through a retrieved document or tool result, so the attacker never needs access to the user's own prompt. Such attacks have already been demonstrated against real systems [1].

Most current defenses treat the model as a black box; they filter inputs, classify prompts before they reach the model, or inspect outputs afterward. This project, however, asks a different question: when a model is about to comply with an injected instruction, is that outcome already visible in its internal state before generation begins? If so, susceptibility is not only a property of the input but partially legible in the model's representations. This carries practical value, since such a signal could support lightweight runtime monitoring, and scientific value, since it bears on how a transformer distinguishes instructions from data.

Rather than asking whether an injection is present in the context, I condition on its presence and ask whether the model's reaction is predictable from internal state. Concretely, I will train linear probes on residual-stream activations at the final prompt token, before any output is produced, and measure how well they predict whether the model actually follows the injected instruction.

2 Background and Related Work

Prompt injection. Greshake et al. [1] introduced indirect prompt injection as an attack class against LLM-integrated applications. Several benchmarks now supply standardized scenarios: AgentDojo [2] provides agentic task suites with injected instructions, InjecAgent [3] targets tool-integrated agents, and BIPIA [4] spans email, web, table, summarization, and code contexts. Wallace et al. [5] attribute the underlying problem to the absence of an instruction hierarchy: models are not trained to treat text differently depending on where it came from.

Probing internal representations. Linear probing [6] tests whether a property is linearly decodable from intermediate activations. Most relevant here, Arditi et al. [7] showed that a model's decision to refuse a harmful request is largely mediated by a single linear direction in the residual stream, one that is present before generation begins and that changes behavior when intervened on. Complying with an injected instruction is the same kind of binary decision (follow this instruction or do not), which motivates the hypothesis that it, too, may be linearly represented.

Activation-level injection detection. Attention Tracker [8] detects injections by observing that they pull attention away from the original instruction. More recent work trains probes on hidden states and reports high AUROC at detecting the presence of indirect injections, including for attacks unseen during training [9]. The target in both cases is a property of the input, not of the model: a probe can succeed simply by recognizing instruction-like text in an unexpected place, which says nothing about whether the model encodes how it will respond. This project removes that shortcut by training only on prompts that already contain an injection.

Circuit-level interpretability. A probe shows that a property is decodable, but not how the model computes it. Circuit tracing goes further: it replaces each block's feedforward layer with a sparse, interpretable stand-in, then traces which features drive a given output [10]; applied to a production model, this has yielded mechanistic accounts of refusal, jailbreaks, and a model concealing a hidden goal [11]. One exploratory study has run the same tooling on jailbreaks and prompt injections, finding separable "format" and "policy" pathways that flip compliance when amplified [12]. A second technique, the Jacobian lens, measures how strongly an activation pushes on the model's output tokens, picking out the small set of concepts a model is prepared to say out loud [13]. Both matter for this project. The Jacobian lens bears on baseline (ii), since it separates what a model represents from what it can report; and recent work traces circuits against a probe direction rather than a token prediction [14], the natural next step once a probe works, and what the stretch phase would pursue (Section 7).

3 Problem Description

Let M be an instruction-tuned language model and let p be a prompt combining a benign user task with untrusted context into which an instruction i has been injected. Under greedy decoding, $M(p)$ is deterministic, so compliance can be defined as a binary label

$$y(p) = \mathbb{1}[\, M(p) \text{ carries out the injected instruction } i \,]$$

Let $h\ell(p)$ denote the residual-stream activation at the final prompt token in layer ℓ, the model's state immediately before it emits its first output token. The relevant question is whether some simple classifier $f\ell$ predicts $y(p)$ from $h\ell(p)$ on held-out prompts, and whether it does so better than classifiers that only see the surface text of p.

Two design choices rule out trivial explanations of a positive result. First, every example contains an injection, so a probe cannot succeed merely by detecting that an attack is present. Second, greedy decoding makes *y(p)* a deterministic function of *p*, whereas under stochastic sampling the same prompt can yield compliance on one run and resistance on the next. That variance is, in principle, unpredictable from a single pre-generation activation vector.

The project addresses four research questions:

RQ1. Do linear probes on residual-stream activations predict injection compliance above chance, and at which layers does the signal emerge?

RQ2. Do such probes outperform baselines that see only the prompt text?

RQ3. Does the signal generalize to attack families held out of training, or is it specific to the templates the probe has already seen?

RQ4. *(Stretch)* Does intervening along the learned direction causally change whether the model complies?

4 Planned Approach

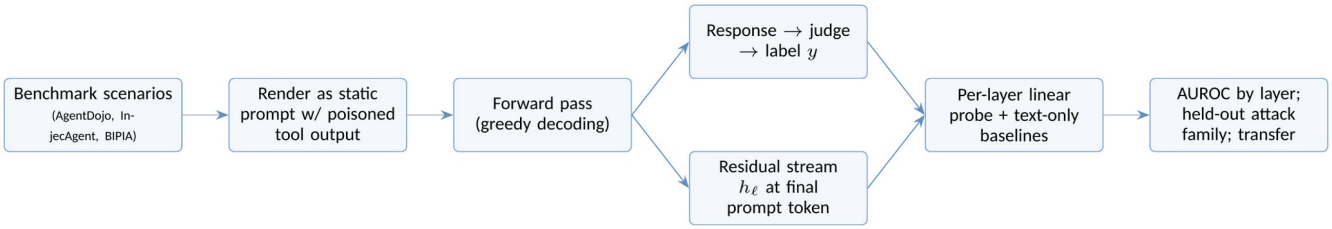
Model and setting. The primary model will be a small open-weight instruction-tuned model (Qwen3-4B-Instruct-2507, with Llama-3.2-3B-Instruct as an alternative), chosen so that full-corpus activation extraction is feasible on a single consumer GPU. I will study indirect injection in a *static context*: each benchmark scenario is rendered as a single prompt containing a poisoned "tool result" or retrieved document, and I check whether the model's response carries out the injected instruction. This keeps the threat model realistic while avoiding the engineering overhead of a live agent loop.

Dataset construction. I will assemble roughly 2,000 injected prompts from AgentDojo, InjecAgent, and BIPIA, spanning at least six attack families (direct imperative, fake system message, urgency framing, role reassignment, delimiter confusion, encoded payload) crossed with several benign task types. A smaller direct-injection set will serve as a contrast condition, and a fallback if the indirect prompts prove too easy or too hard to yield usable label balance.

Labeling. Compliance will be determined automatically, by exact or regular-expression matching against a canary string or target action where the benchmark supports it and an LLM judge otherwise. I will hand-label a pilot set of roughly 200 prompts in full, use those labels to calibrate the automated judge, and report judge agreement before applying the judge to the remaining corpus. The pilot will also identify attack–task combinations whose attack success rate falls in a usable band (roughly 20–80%); combinations the model always resists or always follows carry no predictive signal and will be excluded or reweighted.

Activation extraction and probing. Using HuggingFace Transformers with forward hooks (or TransformerLens/nnsight), I will cache the residual stream at the final prompt token for every layer, in half precision, on the order of a few hundred megabytes for the full corpus. For each layer, I will fit an L_2-regularized logistic regression under 5-fold cross-validation and report AUROC, giving a layer-wise profile of where compliance becomes decodable.

Baselines. To test whether activations contribute anything beyond surface features, I will compare against (i) a TF-IDF logistic-regression classifier over the raw prompt text, (ii) the model's own answer when asked directly whether the context contains a manipulation attempt, and (iii) prompt perplexity. Generalization will be tested by training probes leave-one-attack-family-out, so every evaluation is on an attack type that is absent from training.



Tools and data. Python, PyTorch, HuggingFace Transformers, TransformerLens or nnsight, scikit-learn; AgentDojo, InjecAgent, and BIPIA as prompt sources; for the stretch phase, Anthropic's open-source circuit-tracing library and Neuronpedia's J-lens interface; a single GPU via the Zoo or Colab.

Contingency. If the pilot probes show no decodable signal, the project will pivot at the October 19 checkpoint to characterizing *which prompt-level* features predict compliance, retaining the same dataset and evaluation harness. A negative result for RQ1 — that compliance is not linearly decodable before generation, even though the presence of an injection demonstrably is [9] — would in itself be a reportable finding, speaking directly to the difference between a model registering an attack and acting on it.

5 Schedule and Milestones

Dates	Phase	Milestone
Sep 18 – Sep 27	**Infrastructure**	Activation extraction and caching pipeline working end to end on a toy prompt set.
Sep 28 – Oct 9	**Pilot data**	~200 pilot prompts hand-labeled in full; automated judge calibrated against them; attack success rates measured per attack family.
Oct 12 – Oct 19	**Pilot probes**	Probes at three layers on pilot data. **Go/no-go checkpoint (Oct 19):** proceed with core plan or trigger contingency pivot.
Oct 20 – Oct 25: October recess		
Oct 26 – Nov 6	**Full corpus**	~2,000 prompts generated, judge-labeled, and cached; full per-layer probe sweep with cross-validation.
Nov 9 – Nov 19	**Baselines & generalization**	Text-only, self-report, and perplexity baselines complete; leave-one-attack-family-out results.
Nov 20 – Nov 29: November recess		
Nov 30 – Dec 6	**Stretch / buffer**	If on schedule: steering intervention and attribution analysis of the probe direction. Otherwise reserved as schedule buffer.
Dec 7 – Dec 11	**Writeup**	Final report, poster, project webpage, and public code and data release.

6 Deliverables

1. **Code repository** with the full pipeline (prompt rendering, activation extraction, labeling, probe training, evaluation) released under an open license and available to the Yale community.
2. **Labeled dataset** of injected prompts with per-example compliance labels and metadata, plus scripts and seeds sufficient to regenerate the cached activations.
3. **Attack success rate characterization** for the chosen model by attack family and task type, a standalone result independent of the probing outcome.
4. **Trained probes and results**: per-layer AUROC curves, baseline comparisons, leave-one-attack-family-out generalization.
5. **Final written report** (~12–15 pages) covering methodology, results, limitations, and interpretation.
6. **Project webpage** on the Zoo, plus a poster and final presentation.

7 Future Work

Beyond this semester, the natural first extensions are any stretch goals left unreached: whether probe directions transfer across model families and scales, and whether steering along the learned direction serves as a lightweight runtime defense without costing benign task utility. Further research questions are mechanistic rather than predictive. Attribution graphs built on the probe direction [14] would show which features drive compliance, and the Jacobian lens [13] would show whether a model that registers an injection internally could report it if asked, or whether the signal is legible to the probe but not to the model itself. A live agent loop would carry the same questions into multi-turn settings, where the probe could be queried at each step.

8 References

[1] K. Greshake et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISec, 2023.
[2] E. Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS D&B, 2024.
[3] Q. Zhan et al. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. Findings of ACL, 2024.
[4] J. Yi et al. Benchmarking and Defending Against Indirect Prompt Injection Attacks on LLMs. KDD, 2025.
[5] E. Wallace et al. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208, 2024.
[6] G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. arXiv:1610.01644, 2016.
[7] A. Arditi et al. Refusal in Language Models Is Mediated by a Single Direction. NeurIPS, 2024.
[8] K.-H. Hung et al. Attention Tracker: Detecting Prompt Injection Attacks in LLMs. Findings of NAACL, 2025.
[9] J. Dong et al. Your Agentic LLMs Secretly Encode Indirect Prompt-Injection Exposure in Hidden States. arXiv:2608.02657, 2026.
[10] E. Ameisen et al. Circuit Tracing: Revealing Computational Graphs in Language Models. Transformer Circuits, 2025.
[11] J. Lindsey et al. On the Biology of a Large Language Model. Transformer Circuits, 2025.
[12] M. Fomin. Interpreting Jailbreaks and Prompt Injections with Attribution Graphs. Zenity Labs, 2025.
[13] W. Gurnee et al. Verbalizable Representations Form a Global Workspace in Language Models. Transformer Circuits, 2026.
[14] V. Palit et al. How Do Linear Probes Emerge? A Circuit-Tracing Framework with Concept-Targeted Attribution. arXiv:2608.27510, 2026.